

XPINN Plotting Library — Quick Manual

DIFFICE_jax · examples/render_solver_xpinn_kfac_plots.py

This module is both a command-line script (reproduces the standard figure set from a saved solver) and an importable library of plotting helpers. Everything below is exported in `__all__`. Figures are written to disk as PNG; nothing is shown interactively (Matplotlib runs headless).

1. Making it importable

The module lives under `examples/`, which is not an installed package. Add it to the path once, then import:

```
import sys
sys.path.insert(0, "examples")          # path to the folder, from the repo root
import render_solver_xpinn_kfac_plots as xp
```

```
xp.configure_plot_font()                # apply the shared styling (call once)
```

Run your script from the repository root so the relative path resolves. `configure_plot_font()` falls back to a default font if the bundled one is missing, so it is safe on any machine.

2. What each function does

Function	Purpose
<code>render_from_saved_solver(config_path, solver_dir, tag)</code>	One call: load a trained solver, predict, and write every standard figure. Start here.
<code>save_data_field_comparison(path, data_regions)</code>	u/v/h/s truth-vs-prediction grid. <code>data_regions</code> maps field name -> list of region dicts.
<code>save_inversion_comparison(path, mu_regions, c_regions)</code>	Viscosity (μ) and basal friction (C). Auto-detects real data (NaN targets) -> inferred-only.
<code>save_field_comparison(path, rows, ...)</code>	Generic truth / prediction / error grid, one field per row. The building block.
<code>save_predicted_fields(path, rows, ...)</code>	One predicted field per row, no truth/error columns (for inversion targets).
<code>save_equation_residuals(path, residual_regions)</code>	x- and y-momentum residual maps.
<code>save_x_equation_term_ratios(path, term_regions, bounds_regions)</code>	x-equation term magnitudes relative to driving stress.
<code>save_loss_curve(path, history)</code>	Training loss vs iteration (history is an (N,2) array of step,loss).
<code>tripcolor_regions(ax, regions, key, title, cmap, vmin, vmax)</code>	Low-level: draw one scattered field over regions onto a Matplotlib axis.
<code>field_region(x, y, truth, pred, min_truth=None)</code>	Build a region dict with truth+prediction. truth or pred may be None.
<code>value_region(x, y, value)</code>	Build a region dict holding a single value array (for residual/term plots).

3. The region dict (the data shape helpers expect)

Every high-level helper takes lists of **region dicts**. Build them with `field_region` (truth vs prediction) or `value_region` (single field). Coordinates are in metres.

```
r = xp.field_region(x, y, truth=obs_u, pred=predicted_u)  # -> {x,y,truth,pred,mismatch}
r = xp.field_region(x, y, truth=obs_s, pred=None)         # observed-only (no prediction)
r = xp.value_region(x, y, value=residual_x)              # -> {x,y,value}
```

4. Recipe A — every figure from a saved solver

As a command-line script:

```
python examples/render_solver_xpinn_kfac_plots.py \
    --config <workflow.yaml> --solver-dir <dir> --tag my_run
# figures land in <solver-dir>/plots/
```

Or from Python:

```
xp.render_from_saved_solver("path/to/workflow.yaml", "path/to/solver_dir", "my_run")
```

5. Recipe B — build a custom figure from helpers

```
from pathlib import Path

rows = [
    {"regions": u_regions, "title": "velocity u", "unit": "m/yr", "cmap": "viridis", "key": "u"},
    {"regions": h_regions, "title": "thickness h", "unit": "m", "cmap": "terrain", "key": "h"},
]
stats = xp.save_field_comparison(Path("out/fields.png"), rows, subtitle="My comparison")
# stats -> {"u_rel_mae": ..., "h_rel_mae": ...}

# inferred-only fields (no ground truth), e.g. inversion targets:
xp.save_predicted_fields(Path("out/inferred.png"),
    [{"regions": mu_regions, "title": "inferred mu", "unit": "Pa s", "cmap": "viridis"}])
```

6. Notes for real-data inversions

- **Inversion targets have no ground truth.** mud/alpha2d are NaN in real data. `save_inversion_comparison` detects this and renders inferred-only automatically — no code change.
- **Surface elevation** (s) lives on its own `xd_s/xd_s` grid; if the network has no co-located prediction there, the surface panel renders observed-only.
- **Train with the plain joint-inversion workflow**, never joint-inversion-regression — the latter consumes the NaN targets and produces NaN training.